ProSuite Industrial

CLI & AUTOMATION

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProKey Kommandozeilen-Referenz**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Hardware Entropy CLI
2.0.0	27.02.2026	S. Köhne	Stable	Hex-Output Support, Strict Length Validation & Pipeline Optimization

1 Produktübersicht

Die **ProKey Executable** (`prokey.exe` / `prokey`) ist das hochperformante Command-Line-Interface (CLI) zur Generierung von echter Hardware-Entropie. Im Gegensatz zu softwarebasierten Pseudozufallsgeneratoren (PRNGs) liefert ProKey absolute Zufallswerte, die sich ideal für die Erzeugung kryptografischer Schlüssel (AES, RSA, ECC) oder hochsicherer Session-Tokens eignen.

Kernmerkmale der CLI:

- **Flexible Schlüssellängen:** Native Unterstützung für die industriellen Standards 128, 256, 512 und 1024 Bit.
- **Raw & Hex Formate:** Schlüssel können entweder als speichereffiziente Raw-Binärdaten oder als menschenlesbare, übertragungsfertige Hex-Strings ausgegeben werden.
- **Pipeline-Architektur:** Volle Unterstützung für `stdout`-Piping (`-O -`), um Schlüssel in Unix/Windows-Umgebungen nahtlos an andere Krypto-Tools (wie OpenSSL oder GPG) weiterzureichen.
- **Zero-Trust Memory Wipe:** Nach der Generierung und dem Wegschreiben der Daten wird der reservierte RAM-Speicher der Applikation rigoros mit Nullen überschrieben, um Kaltstart-Attacken (Cold Boot Attacks) zu verhindern.

2 Technische Spezifikationen

2.1 Kommandozeilen-Parameter (CLI)

Die Executable erwartet folgende Syntax: `prokey [OPTIONEN]`

Parameter	Kurzform	Beschreibung	Standardwert
`--output <file>`	`-O`	Pfad zur Ausgabedatei. Bei Angabe von `-` wird direkt in `stdout` geschrieben.	`default.kedc`

`--length <bits>`	`-L`	Schlüssellänge in Bits. Erlaubt: `128`, `256`, `512`, `1024`.	`256`
`--hex`	`-H`	Wandelt die binäre Entropie vor der Ausgabe in einen Hexadezimal-String um.	*Deaktiviert (Raw)*
`--help`	`-h`	Zeigt das Hilfemenü an und beendet das Programm.	-

3 Globales Error-Management (Exit Codes)

Für die nahtlose Integration in Skripte nutzt die ProKey Executable standardisierte Exit-Codes. Alle Statusmeldungen und Fehler werden strikt über `stderr` ausgegeben, damit sie Krypto-Pipelines in `stdout` nicht korrumpieren.

Exit Code	***Fehlerquelle***	***Beschreibung & Behebung***
0	**SUCCESS**	Der Schlüssel wurde erfolgreich generiert und geschrieben.
1	**ERR_PARAM**	Nicht unterstützte Schlüssellänge angefordert. Korrigieren Sie den Wert auf 128, 256, 512 oder 1024.
1	**ERR_MEM**	RAM-Allokation fehlgeschlagen (sehr selten, tritt nur bei kritischem Speichermangel auf).
1	**ERR_IO**	Datei konnte nicht geöffnet/beschrieben werden (z. B. fehlende Schreibrechte im Zielverzeichnis).

4 Linux / Bash Integration

Unter Linux ist ProKey das ideale Werkzeug, um Krypto-Container (z. B. LUKS) sicher zu initialisieren oder Automatisierungsprozesse mit Entropie zu versorgen.

4.1 Beispiel 1: Erstellung eines AES-256 Masterkeys (Raw)

```bash
#!/bin/bash
KEY_FILE="/etc/proed/master_key.bin"

echo "Generiere Hardware-Schlüssel..."
./prokey -L 256 -O "$KEY_FILE"

if [ $? -eq 0 ]; then
    chmod 400 "$KEY_FILE" # Nur Root darf lesen
    echo "Schlüssel erfolgreich gesichert."
else
    echo "Kritischer Fehler bei der Schlüsselerstellung!"
    exit 1
fi
```

4.2 Beispiel 2: Pipeline-Generierung für OpenSSL

Anstatt den Schlüssel erst auf die Festplatte zu schreiben (was ein Sicherheitsrisiko sein kann), generieren wir ihn on-the-fly und übergeben ihn als Hex-String direkt an einen anderen Prozess:

```bash
# Nutzt ProKey als Hex-RNG-Quelle für Krypto-Prozesse
SECRET=$(./prokey -L 512 -H -O -)
echo "Ihr temporärer Session-Token: $SECRET"
```

5 Windows PowerShell Integration

Auf Windows-Systemen ist die CLI gegen ein bekanntes Pipeline-Problem (`\n` zu `\r\n` Konvertierung in `stdout`) immunisiert, indem der Stream explizit im binären Modus betrieben wird (`_O_BINARY`).

5.1 Beispiel: Erzeugung eines API-Tokens im Hex-Format

```
Write-Host "Initialisiere Hardware-Entropie..."

# Generiere einen 1024-Bit langen Schlüssel als Hex-Datei
$process = Start-Process -FilePath "prokey.exe" -ArgumentList "-L 102

if ($process.ExitCode -eq 0) {
    Write-Host "API Token erfolgreich erstellt." -ForegroundColor Gre
} else {
    Write-Host "Hardware-Fehler bei der Schlüsselerzeugung (Code: $($
}
```

6 Best Practices & Sicherheitsrichtlinien

- **Raw vs. Hex:** Nutzen Sie das Raw-Format (Standard), wenn der Schlüssel lokal auf der Festplatte als Datei (`.bin` / `.key`) zur maschinellen Nutzung gespeichert wird. Nutzen Sie den Parameter `-H` (`--hex`), wenn der Schlüssel über Textkanäle (JSON, REST APIs, Konfigurationsdateien) übertragen oder durch Menschen abgetippt werden muss.
- **Pipeline Sicherheit:** Das direkte Pipen in `stdout` (`-O -`) umgeht die Festplatte komplett. Dies ist die sicherste Methode, umlüssel in RAM-Disks oder direkt in laufende Applikationen zu injizieren.
- **Keine Konsolen-Verschmutzung:** Alle Status-Meldungen wie `[+] Generated 256-Bit Key` werden durch die Engine immer in `stderr` geschrieben. Das bedeutet, Sie können den Aufruf `> key.txt` problemlos nutzen, ohne dass der eigentliche Schlüssel durch Textnachrichten korrumpiert wird.

© 2026 Sascha Köhne	✉️ **Contact: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**